PYTHON KURSU • HAFTA 5 • GÜN 1

Fonksiyonlar

Ders 9 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- def ile kendi fonksiyonlarınızı tanımlayıp çağırabileceksiniz.
- Parametre, argüman ve return kavramlarını doğru kullanabileceksiniz.
- Varsayılan değerli ve isimli argümanlarla esnek fonksiyonlar yazabileceksiniz.
- Yerel ve global değişken (kapsam) farkını açıklayabileceksiniz.
- try/except ile hatalı girişte programın çökmesini önleyebileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 8’in tekrarı
0:10–1:25	75 dk	Blok 1: neden fonksiyon • def • parametre ve return • varsayılan değerler • kapsam
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: try/except • güvenli giriş • hesaplama ve metin temizleme fonksiyonları • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 8'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Ekranda ne yazar?

```
t = 0
for i in range(1, 4):
    t += i
print(t)
```

SORU 2 • Ekranda ne yazar?

```
for x in [1, 2, 3, 4]:
    if x == 3:
        break
    print(x)
```

SORU 3 • Ekranda ne yazar?

```
for x in [1, 2, 3]:
    if x == 2:
        continue
    print(x)
```

SORU 4 • Neden durmuyor?

```
i = 0
while i < 3:
    print(i)
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev 1: sınıf raporu (kısaltılmış)

```
ogrenciler = ["Ayşe", "Can", "Ece"]
notlar = [88, 45, 72]
for ad, puan in zip(ogrenciler, notlar):
    if puan ⩾ 85:
        harf = "BA+"
    elif puan ⩾ 60:
        harf = "CC-BB"
    else:
        harf = "FF-FD"
    print(f"{ad:<6}{puan:>4}  {harf}")
```

```
ÇIKTI
Ayşe    88  BA+
Can     45  FF-FD
Ece     72  CC-BB
```

Harf notu kodunu başka bir programda da kullanmak istesek? **Kopyalamak yerine bir kez yazıp adıyla çağıracağız.**

BLOK 1 • 0:10–1:25 • 75 DK

Kendi fonksiyonlarımız

15 dk
def ve çağırma

25 dk
Parametre ve return

15 dk
Varsayılan değerler

20 dk
Kapsam

BLOK 1 • DEF VE ÇAĞIRMA • 15 DK

def ile tanımla, adıyla çağır

```
def selamla():
    print("Merhaba!")
    print("Kursa hoş geldiniz.")

selamla()
selamla()
```

```
ÇIKTI
Merhaba!
Kursa hoş geldiniz.
Merhaba!
Kursa hoş geldiniz.
```

Benzetme: yemek tarifi

Tarifi deftere bir kez yazarsınız (def). Her pişirmek istediğinizde tarifin adını söylemeniz yeterli (çağırma).

BLOK 1 • DEF VE ÇAĞIRMA • 15 DK

Neden fonksiyon yazarız?

1 • TEKRAR YOK

Bir kez yaz, çok kullan

Aynı kodu kopyalamak yerine adıyla çağırın. Kuralı değiştirmek gerekirse tek yeri düzeltirsiniz.

2 • OKUNURLUK

Kod bir hikâye gibi okunur

kdv_ekle(fiyat) satırı, içindeki hesaplamayı bilmeden de ne yaptığını anlatır.

3 • TEST

Parça parça denenir

Her fonksiyon küçük örneklerle ayrı ayrı test edilebilir; hata aramak kolaylaşır.

4 • PAYLAŞIM

Başkaları da kullanır

Ders 10'da göreceğiniz kütüphaneler, başkalarının yazıp paylaştığı fonksiyon koleksiyonlarıdır.

BLOK 1 • PARAMETRE VE RETURN • 25 DK

Parametre: fonksiyonun aldığı girdi

```python
def selamla(ad):
    print(f"Merhaba {ad}!")

def kdv_goster(fiyat, oran):
    print(f"{fiyat} TL + %{oran} KDV = {fiyat * (1 + oran / 100):.2f} TL")

selamla("Ayşe")
selamla("Mehmet")
kdv_goster(100, 20)
kdv_goster(250, 10)
```

```
ÇIKTI
Merhaba Ayşe!
Merhaba Mehmet!
100 TL + %20 KDV = 120.00 TL
250 TL + %10 KDV = 275.00 TL
```

Terim
parametre
argüman

Nerede?
tanımdaki değişken adı
çağırırken verilen değer

Örnek
def selamla(**ad**):
selamla(**"Ayşe"**)

BLOK 1 • PARAMETRE VE RETURN • 25 DK

return: sonucu çağırana geri ver

```python
def kdv_ekle(fiyat, oran):
    return fiyat * (1 + oran / 100)

kitap = kdv_ekle(100, 10)
kahve = kdv_ekle(85, 20)
print(f"Kitap: {kitap:.2f} TL")
print(f"Toplam: {kitap + kahve:.2f} TL")
print(f"{kdv_ekle(1000, 20):.2f}")
```

```
ÇIKTI
Kitap: 110.00 TL
Toplam: 212.00 TL
1200.00
```

Benzetme: meyve sıkacağı (Ders 4)

Portakalı koyarsınız (argüman), makine sıkar (fonksiyon), bardağa meyve suyu gelir (return). O suyu içmek, saklamak ya da başka tarife eklemek size kalmış.

BLOK 1 • PARAMETRE VE RETURN • SIK YAPILAN HATA

print gösterir, return geri verir

print ile

```
def kare_yaz(x):
    print(x * x)

sonuc = kare_yaz(4)
print("Sonuç:", sonuc)
```

```
ÇIKTI
16
Sonuç: None
```

return ile

```
def kare(x):
    return x * x

sonuc = kare(4)
print("Sonuç:", sonuc)
```

```
ÇIKTI
Sonuç: 16
```

Kural: Fonksiyon bir **hesap** yapıyorsa sonucu **return** etsin; yazdırmaya ana program karar versin.

BLOK 1 • PARAMETRE VE RETURN • 25 DK

Erken return ve birden fazla değer

```python
def harf_notu(puan):
    if puan ⩾ 85:
        return "BA"
    if puan ⩾ 70:
        return "CC"
    if puan ⩾ 50:
        return "FD"
    return "FF"

def istatistik(sayilar):
    return min(sayilar), max(sayilar), sum(sayilar) / len(sayilar)

print(harf_notu(92), harf_notu(73), harf_notu(30))
en_kucuk, en_buyuk, ort = istatistik([70, 85, 92])
print(en_kucuk, en_buyuk, round(ort, 1))
```

```
ÇIKTI
BA CC FF
70 92 82.3
```

BLOK 1 • VARSAYILAN DEĞERLER • 15 DK

Varsayılan değerler ve isimli argümanlar

```python
def fiyat_hesapla(tutar, kdv=20, indirim=0):
    indirimli = tutar * (1 - indirim / 100)
    return round(indirimli * (1 + kdv / 100), 2)

print(fiyat_hesapla(100))
print(fiyat_hesapla(100, 10))
print(fiyat_hesapla(100, indirim=15))
print(fiyat_hesapla(indirim=15, tutar=100, kdv=1))
```

```
ÇIKTI
120.0
110.0
102.0
85.85
```

Çağrı	tutar	kdv	indirim	Sonuç
`fiyat_hesapla(100)`	100	20 (varsayılan)	0 (varsayılan)	120.0
`fiyat_hesapla(100, 10)`	100	10	0	110.0
`fiyat_hesapla(100, indirim=15)`	100	20	15	102.0

BLOK 1 • VARSAYILAN DEĞERLER • 15 DK

Belge metni: fonksiyonunuzu açıklayın

```
def vki(kilo, boy):
    """Kilo (kg) ve boy (m) alır, vücut kitle indeksini döndürür."""
    return kilo / boy ** 2

print(round(vki(70, 1.75), 2))
print(vki.__doc__)
```

```
ÇIKTI
22.86
Kilo (kg) ve boy (m) alır, vücut kitle indeksini döndürür.
```

help(vki) yazan ya da VS Code’da fonksiyon adının üzerine gelen herkes bu açıklamayı görür.

BLOK 1 • KAPSAM • 20 DK

Kapsam: yerel değişkenler dışarıdan görünmez

```
sepet_toplam = 0

def ekle(fiyat):
    adet = 1
    return sepet_toplam + fiyat * adet

print(ekle(50))
print(sepet_toplam)
print(adet)
```

```
ÇIKTI
50
0
NameError: name 'adet' is not defined. Did you mean:
  'anext'?
```

Benzetme: otel odası

Odadaki minibar (yerel değişken) yalnızca o odada kullanılır; check-out'ta temizlenir. Lobideki çay (global) herkese açıktır; odadan bakılabilir ama odadan değiştirilmez.

Değişken	Kapsam
`sepet_toplam`	global (her yerden okunur)
`fiyat, adet`	yerel (yalnızca ekle içinde)

BLOK 1 • KAPSAM • 20 DK

Global bir değişkeni fonksiyondan değiştirmek

Çalışmaz

```
sayac = 0

def artir():
    sayac += 1

artir()
```

```
HATA
UnboundLocalError: cannot access local
 variable 'sayac'
where it is not associated with a value
```

Doğru yol: parametre al, sonucu return et

```
sayac = 0

def artir(deger):
    return deger + 1

sayac = artir(sayac)
sayac = artir(sayac)
print(sayac)
```

```
ÇIKTI
2
```

Fonksiyonlarda hata mesajları

Hatalı kod	Hata mesajı	Çözüm
`selamla() # tanımdan önce`	`NameError: name 'selamla' is not defined`	Önce def, sonra çağrı
`selamla() # ad verilmedi`	`TypeError: selamla() missing 1 required positional argument: 'ad'`	Argümanı verin: selamla("Ali")
`selamla("Ali", "Veli")`	`TypeError: selamla() takes 1 positional argument but 2 were given`	Parametre sayısı kadar argüman
`def f(a=1, b):`	`SyntaxError: parameter without a default follows parameter with a default`	Varsayılanlı parametreler sona
`return 5 # fonksiyon dışında`	`SyntaxError: 'return' outside function`	return yalnızca fonksiyon içinde

1:25–1:40 • 15 DK

Mola

Moladan sonra: Kullanıcı “elli” yazınca int() programı çökertiyor.
Çökmeden nasıl “tekrar deneyin” deriz?

BLOK 2 • 1:40–2:55 • 75 DK

Hataları yakalamak ve fonksiyonlarla uygulamalar

20 dk
try/except

10 dk
Güvenli giriş

25 dk
Uygulamalar

20 dk
Alıştırmalar

BLOK 2 • TRY/EXCEPT • 20 DK

try/except: dene, hata olursa yakala

```
giris = input("Bir sayı girin: ")
try:
    sayi = int(giris)
    print("Karesi:", sayi ** 2)
except ValueError:
    print(f"'{giris}' bir sayı değil.")
print("Program devam ediyor.")
```

```
GİRİŞ: 12
Bir sayı girin: 12
Karesi: 144
Program devam ediyor.
```

```
GİRİŞ: elli
Bir sayı girin: elli
'elli' bir sayı değil.
Program devam ediyor.
```

BLOK 2 • TRY/EXCEPT • 20 DK

Farklı hatalara farklı tepki: except, else, finally

```
def bol(a, b):
    try:
        sonuc = a / b
    except ZeroDivisionError:
        print("Sıfıra bölünemez.")
        return None
    except TypeError:
        print("Yalnızca sayılar bölünebilir.")
        return None
    else:
        return round(sonuc, 2)
    finally:
        print("bol() bitti.")

print(bol(10, 3))
print(bol(10, 0))
print(bol("10", 2))
```

```
ÇIKTI
bol() bitti.
3.33
Sıfıra bölünemez.
bol() bitti.
None
Yalnızca sayılar bölünebilir.
bol() bitti.
None
```

BLOK 2 • GÜVENLI GIRIŞ • 10 DK

Bir kez yaz, her yerde kullan: güvenli sayı alma

```python
def sayi_al(mesaj, en_az=None, en_cok=None):
    while True:
        giris = input(mesaj).strip().replace(",", ".")
        try:
            sayi = float(giris)
        except ValueError:
            print("Lütfen bir sayı yazın.")
            continue
        if en_az is not None and sayi < en_az:
            print(f"En az {en_az} olmalı.")
        elif en_cok is not None and sayi > en_cok:
            print(f"En çok {en_cok} olabilir.")
        else:
            return sayi

puan = sayi_al("Not (0-100): ", 0, 100)
print("Kaydedildi:", puan)
```

```
ÇIKTI
Not (0-100): elli
Lütfen bir sayı yazın.
Not (0-100): 120
En çok 100 olabilir.
Not (0-100): 87,5
Kaydedildi: 87.5
```

BLOK 2 • TRY/EXCEPT • SIK YAPILAN HATA

Her hatayı sessizce yutmayın

YANLIŞ: çıplak except

```
notlar = [70, 80]
try:
    ortalama = sum(notlar) / len(notlar)
    print("Ortalama:", ortalma)
except:
    print("Bir şeyler ters gitti.")
```

```
ÇIKTI
Bir şeyler ters gitti.
```

DOĞRU: belirli hata türü

```
notlar = []
try:
    ortalama = sum(notlar) / len(notlar)
    print("Ortalama:", ortalama)
except ZeroDivisionError:
    print("Hiç not girilmemiş.")
```

```
ÇIKTI
Hiç not girilmemiş.
```

BLOK 2 • GERÇEK HAYATTAN UYGULAMA 1 • 10 DK

Taksitli alışveriş hesaplayıcı

fiyat.py

```python
def indirim_uygula(tutar, oran=0):
    return tutar * (1 - oran / 100)

def kdv_ekle(tutar, kdv=20):
    return tutar * (1 + kdv / 100)

def taksit_hesapla(tutar, taksit=1):
    return tutar / taksit

def tl(tutar):  # 1234.5 → "1.234,50 TL" (tam hali notlarda)
    ...

toplam = kdv_ekle(indirim_uygula(12500, 15))
aylik = taksit_hesapla(toplam, 6)
```

ÇIKTI

```
Liste fiyatı : 12.500,00 TL
İndirimli    : 10.625,00 TL
KDV dahil    : 12.750,00 TL
6 taksit     : 2.125,00 TL / ay
```

Kısaltılmış: tam hali notlarda ve .py dosyasında.

BLOK 2 • UYGULAMA 1 • SATIR SATIR

Fonksiyonlar zincir gibi birbirine bağlanır

Adım	Çağrı	Dönen değer
1	`indirim_uygula(12500, 15)`	`10625.0`
2	`kdv_ekle(10625.0)`	`12750.0`
3	`taksit_hesapla(12750.0, 6)`	`2125.0`
4	`tl(2125.0)`	`"2.125,00 TL"`

İç içe çağrı nasıl okunur?

İçteki parantez önce çalışır: kdv_ekle(indirim_uygula(12500, 15)) → önce indirim, sonucu kdv_ekle'ye argüman olur.

BLOK 2 • GERÇEK HAYATTAN UYGULAMA 2 • 10 DK

Form verisi temizleme: Ders 3'ün sözünü tutuyoruz

temizle.py

```
def tr_buyuk(metin):
    return metin.replace("i", "İ").replace("ı", "I").upper()

def tr_kucuk(metin):
    return metin.replace("I", "ı").replace("İ", "i").lower()

def ad_duzelt(ad):
    duzgun = []
    for k in ad.split():
        duzgun.append(tr_buyuk(k[0]) + tr_kucuk(k[1:]))
    return " ".join(duzgun)

def telefon_temizle(tel):  # rakam dışını at (tam hali notlarda)
    ...
```

ÇIKTI

```
İlker Işık        05321112233
Ayşe Nur Yılmaz   905054445566
Irmak             05427778899
```

BLOK 2 • UYGULAMA 2 • SATIR SATIR

“ışık” kelimesi nasıl “Işık” oldu?

Adım	İşlem	Sonuç
1	`k[0] → "ı"`	ilk harf
2	`tr_buyuk("ı") → replace("ı", "I") → upper()`	"I"
3	`k[1:] → "şık"`	kalan harfler
4	`tr_kucuk("şık") → lower()`	"şık"
5	`"I" + "şık"`	"Işık"

Neden Python’un title() yetmiyor?

Python dil bağımsız çalışır: “ı”nın büyüğünü “I”, “i”nin büyüğünü de “I” sanır. Türkçeye özgü kural için kendi fonksiyonumuzu yazdık; artık her projede tek satırla kullanabiliriz.

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 20 DK

Alıştırma 1 ve 2

KOLAY • 4 dk

1 • Dikdörtgen

dikdortgen(en, boy) fonksiyonu alanı ve çevreyi BİRLİKTE döndürsün.

- alan, cevre = dikdortgen(5, 3)
- Sonucu yazdırın: Alan 15, Çevre 16

KOLAY-ORTA • 5 dk

2 • Kelime sayacı

kelime_say(metin, en_az=1) fonksiyonu, metinde en_az harften uzun ya da eşit kelimelerin sayısını döndürsün.

- kelime_say("bugün hava çok güzel") → 4
- kelime_say("bugün hava çok güzel", 4) → 3

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 20 DK

Alıştırma 3 ve 4

ORTA • 5 dk

3 • Hata avı

Üç hata var: çalıştırın, mesajları okuyun, düzeltin.

```
def ortalama(sayilar):
    toplam = sum(sayilar)
    print(toplam / len(sayilar))

sonuc = ortalama()
print("Ortalama:", sonuc + 1)
```

ZOR • 6 dk

4 • Güvenli hesap makinesi

hesapla(a, b, islem) fonksiyonu “+ - * /” işlemlerini yapsın:

- Bilinmeyen işlemde None döndürsün
- Sıfıra bölmede try/except ile None döndürsün
- Kullanıcıdan iki sayı ve işlem alın (sayılar için Bölüm 12’deki sayi_al)

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1 ve 2

1 • Dikdörtgen

```
def dikdortgen(en, boy):
    alan = en * boy
    cevre = 2 * (en + boy)
    return alan, cevre

alan, cevre = dikdortgen(5, 3)
print(f"Alan {alan}, Çevre {cevre}")
```

```
Alan 15, Çevre 16
```

2 • Kelime sayacı

```
def kelime_say(metin, en_az=1):
    sayac = 0
    for k in metin.split():
        if len(k) ⩾ en_az:
            sayac += 1
    return sayac

print(kelime_say("bugün hava çok güzel"))
print(kelime_say("bugün hava çok güzel", 4))
```

```
4
3
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 3: hata avı

```
def ortalama(sayilar):
    if not sayilar:
        return 0
    return sum(sayilar) / len(sayilar)

sonuc = ortalama([70, 80, 90])
print("Ortalama:", sonuc + 1)
```

```
Ortalama: 81.0
```

1. **TypeError (eksik argüman):** ortalama([70, 80, 90])
2. **TypeError (NoneType + int):** print yerine return
3. **Gizli hata:** boş liste ZeroDivisionError verirdi → erken return 0

Çözüm 4: güvenli hesap makinesi

```
def hesapla(a, b, islem):
    if islem == "+":
        return a + b
    elif islem == "-":
        return a - b
    elif islem == "*":
        return a * b
    elif islem == "/":
        try:
            return a / b
        except ZeroDivisionError:
            return None
    return None

print(hesapla(8, 2, "/"), hesapla(8, 0, "/"), hesapla(8, 2, "%"))
```

```
4.0 None None
```

Sırasıyla: 8/2, sıfıra bölme, bilinmeyen işlem.

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- **def ad(parametre):** tanımlar; **ad(argüman)** çağırır.
- **return** sonucu geri verir; return yoksa fonksiyon None döndürür.
- Varsayılan değer **kdv=20**; isimli argüman **indirim=15**.
- Fonksiyon içindeki değişkenler yereldir; değeri parametreyle al, return ile ver.
- **try/except HataTürü** ile hatayı yakala; çıplak except yazma.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Not sistemi fonksiyonları

- harf_notu(puan): Ders 7'nin tam ölçeği
- ortalama(notlar): boş listede 0
- Ders 8 Ödev 1'i bu iki fonksiyonla yeniden yazın

2 • ZORUNLU

Birim çevirici

Varsayılan parametreli üç fonksiyon: km_mil(km), c_f(derece), kg_lb(kg). Kullanıcıya menü gösterin; giriş için sayi_al ve try/except kullanın.

3 • BONUS

Şifre kontrol fonksiyonu

Ders 7 Alıştırma 4'teki kuralları sifre_kontrol(sifre) fonksiyonuna taşıyın; eksik kuralların LİSTESİNİ döndürsün (boş liste = güçlü şifre).

Teslim: ders09_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 5 • GÜN 2

Terminal, pip ve modüller

Kendi fonksiyonlarımızı yazdık; şimdi başkalarının yazdıklarını kullanacağız:

- Terminal ve temel komutlar
- pip ile paket kurma, sanal ortam (venv)
- random, datetime modülleri • kendi modülümüz
- Dosya okuma ve yazma

Önizleme

```
import random
zar = random.randint(1, 6)
print("Zar:", zar)
```

```
Zar: 3
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
Fonksiyon tanımlamak (Türkçe eğitim)	docs.python.org/tr/3/tutorial/controlflow.html
Varsayılan ve isimli argümanlar	docs.python.org/3/tutorial/controlflow.html
Hatalar ve try/except	docs.python.org/3/tutorial/errors.html
Yerleşik hata türleri	docs.python.org/3/library/exceptions.html